

1 Solution — GIAM 2.1

Problem 1

1.1 Problem

Design a digital logic circuit (using **and**, **or**, and **not** gates) that implements an *exclusive or*.

1.2 What Exclusive-Or Means

The exclusive or of A and B , written $A \oplus B$, is true when **exactly one** of A, B is true — that is, when they *differ*. It excludes the case where *both* are true, the one case where ordinary (inclusive) “or” and “exclusive or” disagree:

A	B	$A \vee B$ (inclusive)	$A \oplus B$ (exclusive)
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	0

Table 1.1

1.3 The Boolean Expression

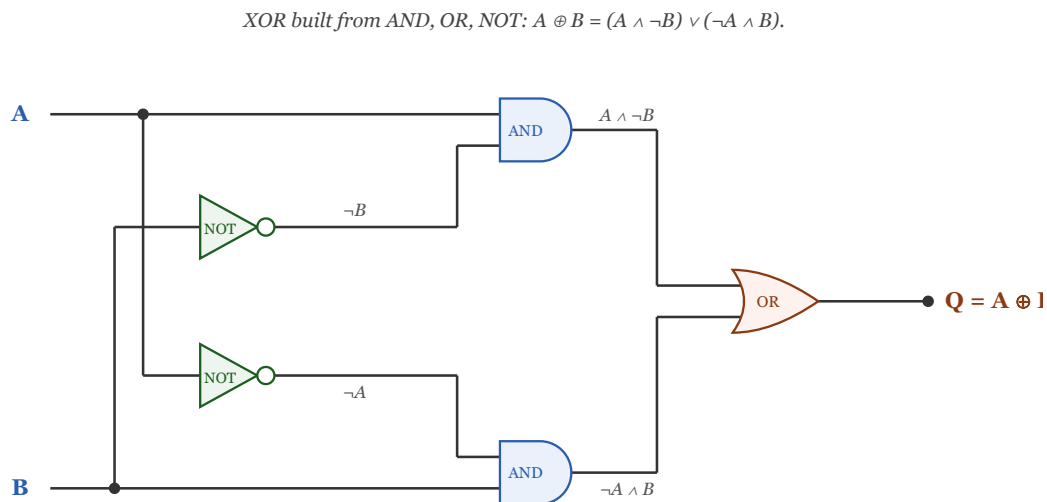
Read the two “output 1” rows of the XOR column and OR them together. Row $(0, 1)$ is “ A false and B true,” row $(1, 0)$ is “ A true and B false.” Hence

$$A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B).$$

This uses only **not** ($\neg$), **and** ($\wedge$), and **or** ($\vee$), as required.

1.4 The Circuit

Build it left to right: negate each input, form the two “one-true-one-false” AND terms, then OR them:



Q is TRUE exactly when A and B differ: one input on, the other off. 5 gates total (2 NOT, 2 AND, 1 OR).

Figure 1.1: A digital logic circuit. Inputs A (top) and B (bottom) each split: A feeds directly into the top AND gate and also into a NOT gate producing not-A; B feeds directly into the bottom AND gate and also into a NOT gate producing not-B. The top AND gate computes A and not-B; the bottom AND gate computes not-A and B. Both AND outputs feed an OR gate, whose output $Q = A \text{ xor } B$. Five gates total: 2 NOT, 2 AND, 1 OR. Q is true exactly when A and B differ.

Signal flow.

1. $\neg A$ and $\neg B$ come from the two NOT gates.
2. The top AND gate forms $A \wedge \neg B$ (true only for $A=1, B=0$).
3. The bottom AND gate forms $\neg A \wedge B$ (true only for $A=0, B=1$).
4. The OR gate outputs 1 when *either* AND fires — i.e. exactly when A and B differ. That is $Q = A \oplus B$.

1.5 Remark — Verifying the Circuit Against the Truth Table

Tracing all four input combinations confirms the design:

A	B	$A \wedge \neg B$	$\neg A \wedge B$	$\text{OR} = Q$
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

Table 1.2

The Q column matches the XOR column exactly. Note the two AND terms are never both 1 at once, so the OR here behaves like an *exclusive* combination of two mutually exclusive cases.

1.6 Remark — An Equivalent Design: $(A \vee B) \wedge \neg(A \wedge B)$

The “sum-of-products” form above is the most direct, but XOR also equals

$$A \oplus B = (A \vee B) \wedge \neg(A \wedge B),$$

read as “ A or B , but **not both**.” This phrasing matches the English definition of exclusive-or almost word-for-word, and it also uses five gates (one OR, one AND, one NOT on the $A \wedge B$, plus the AND and OR). The two circuits compute the identical function — a concrete instance of the logical equivalences (a De Morgan / distribution rearrangement) that Chapter 2 studies. Which to prefer is a matter of taste or of which gates are cheap in a given technology.

1.7 Remark — Why XOR Needs More Than One Gate

There is no single **and/or/not** gate that computes XOR: those three are the building blocks, and XOR is a *composite*. This is exactly why XOR (and its complement XNOR) are usually given their own gate symbol in practice — they are common enough to be

worth packaging. The reason XOR cannot be a single AND/OR/NOT is structural: AND and OR are *monotone* (turning an input from 0 to 1 can never turn the output from 1 to 0), whereas XOR is **not** monotone — raising B from 0 to 1 flips the output both ways depending on A . Capturing that non-monotone behavior forces at least one NOT and a combination of gates.

1.8 Remark — Everything Reduces to NAND (Gate Universality)

A deeper fact lurking behind this exercise: **and**, **or**, **not** are not even the minimal toolkit. Every one of them — and hence XOR and *any* Boolean function — can be built from a single gate type, the **NAND** (not-and), or alternatively from **NOR** alone. For instance $\neg A = A \text{ NAND } A$, and $A \wedge B = \neg(A \text{ NAND } B)$. This *functional completeness* of NAND is why real chips are often fabricated almost entirely from NAND gates. The present problem is the first rung of that ladder: it shows a genuinely new operation (XOR) emerging from a small fixed set of primitives — the logical-gates analogue of how all of arithmetic is built from a few axioms, a theme Chapter 2 develops for logic itself.

1.9 Remark — XOR Is Addition Modulo 2

Comparing the truth table to arithmetic, $A \oplus B$ is exactly $(A + B) \bmod 2$: $0+0=0$, $0+1=1$, $1+0=1$, and $1+1 = 2 \equiv 0$. So XOR is “carry-less” binary addition of one bit — which is precisely why XOR gates sit at the heart of **binary adders** (the sum bit of a half-adder is $A \oplus B$, the carry bit is $A \wedge B$). It also makes XOR the workhorse of parity checks and of stream ciphers, where X-ORing with a key bit is its own inverse ($(A \oplus K) \oplus K = A$). A humble five-gate circuit turns out to be the atom of both binary arithmetic and a good deal of cryptography.